

UE24CS352A Machine Learning – Mini-Project

Crowdfunding: Predicting Kickstarter Project Success

NAME : Mithun S

SRN : PES2UG24AM208

1. Problem Statement

Given only information available **at launch** (funding goal, campaign duration, category, country, currency, launch date, and the project name and short blurb), predict whether a Kickstarter project will reach its funding goal (successful) or not (failed). This is a binary classification task. Such a predictor can help creators set realistic goals and help the platform identify promising projects. We follow the approach of Khosla, Reinecke and Wittenbrink (Stanford), *Crowdfunding: Predicting Kickstarter Project Success*, at a smaller scale.

2. Dataset

We use the Web Robots monthly Kickstarter scrapes (the source used in the reference paper), taking the six most recent monthly snapshots. We keep only completed campaigns (successful or failed), remove duplicates across months (same name, blurb, launch date and deadline), convert goals to USD using the recorded exchange rate, and drop invalid durations. Post-launch fields such as backers and amount pledged are excluded to avoid leakage.

- **199,233** projects after cleaning; **63%** successful, 37% failed.
- Stratified 70/30 split: **139,463** training and **59,770** test projects.
- Raw fields used: goal, deadline, launch time, category and parent category, country, currency, name, blurb.

3. Approach

Features (17). Eight numeric features: log goal, goal in USD, duration in days, blurb length and word count, name length, launch month and launch weekday. Eight categorical-derived features: for category, parent category, country and currency, an *out-of-fold target-mean encoding* (each row is encoded using only other folds, to prevent leakage) and the project's log goal minus the mean log goal of its level. One text feature: the success probability from a Multinomial Naive Bayes classifier on the TF-IDF of name + blurb, also computed out-of-fold.

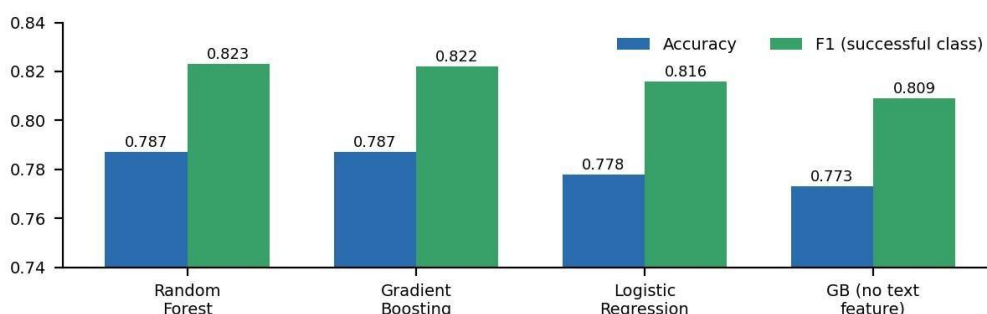
Models. Logistic Regression (standardised inputs, L2, $C = 0.1$), Random Forest (200 trees, max depth 30, min leaf 10) and Gradient Boosting (300 iterations, learning rate 0.1, 64 leaves per tree). Balanced class weights are used for all models. As in the paper, we also run an ablation of Gradient Boosting without the text feature.

4. Implementation Overview

The project is written in Python (pandas, scikit-learn, matplotlib) and hosted in a GitHub repository: `src/data.py` loads and cleans the CSVs; `src/features.py` builds the features; `src/train.py` trains and evaluates the models and saves the best one; `src/app.py` is a Streamlit demo that takes a project's details and returns its predicted probability of success. Running `python src/train.py` reproduces all results below.

5. Results (test set, 59,770 projects)

Model	Accuracy	F1	Precision (succ.)	Precision (failed)	Recall (succ.)	Recall (failed)
Random Forest	0.787	0.823	0.869	0.678	0.782	0.795
Gradient Boosting	0.787	0.822	0.874	0.675	0.776	0.806
Logistic Regression	0.778	0.816	0.860	0.669	0.777	0.780
GB without text feature	0.773	0.809	0.869	0.655	0.756	0.802



6. Conclusions

- Random Forest and Gradient Boosting perform best and are essentially tied (accuracy 0.787, F1 about 0.82); we deploy the Random Forest in the demo. Logistic Regression is close behind (0.778), suggesting most of the signal comes from goal and category.
- The name/blurb text feature gives a small but consistent gain for Gradient Boosting (accuracy 0.773 to 0.787, F1 0.809 to 0.822), in line with the paper's finding that text adds only a little on top of launch metadata.
- All models are weaker on failed projects (precision about 0.68 vs 0.87 for successful), so some failures are predicted as successes.
- Our best accuracy (0.787) is somewhat below the paper's 0.83. Likely reasons are a smaller and different time window, fewer text encodings (no LDA, Word2Vec or sentiment) and no extensive grid search. Future work: hyperparameter tuning with cross-validation, BERT-based text features, and image or video features.